

RED-BLACK TREES

Michael Tsai

~~2026/5/12~~

2026/5/12

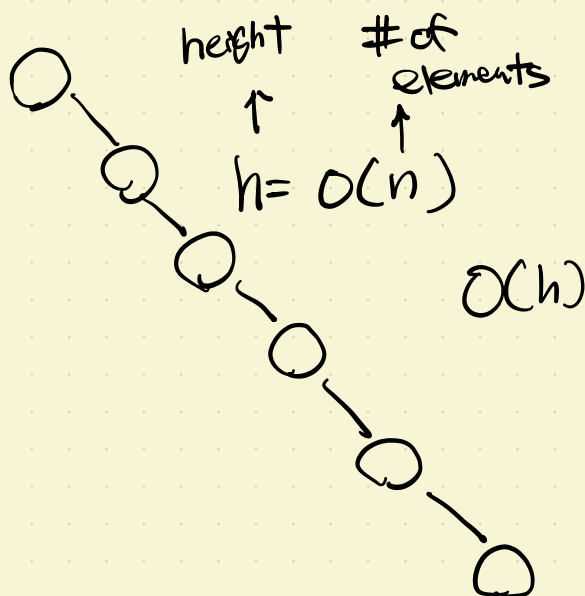
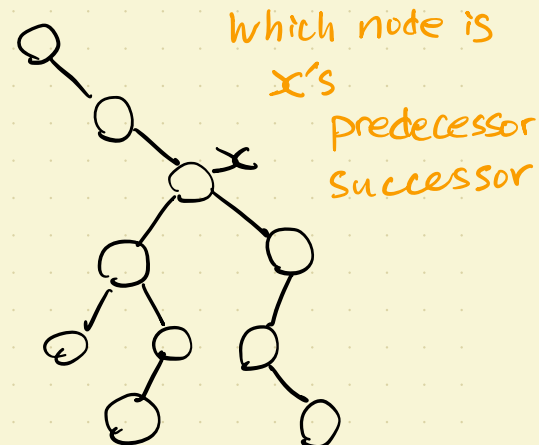
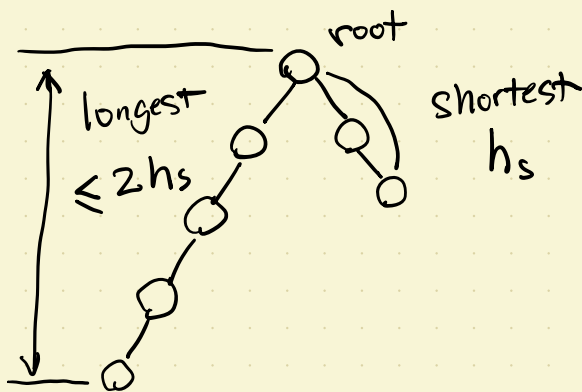
Red-Black Trees $\Rightarrow$ a type of binary search tree

- A **balanced** tree:
Guarantees that the simple path from root to any leaf is **no more than twice** as long as the path from root to any other such leaf
- Roughly balanced — operations can all be done in $O(\log n)$
- Supported operations:
 - Search for an element
 - Insert an element
 - Delete an element
 - Find max or min element
 - Find the **successor** or **predecessor** of an element

Why can they
all be done
in $O(\log n)$? Slide #10

how?

how?



Red-Black Trees

- Each node is assigned a color: **red** or **black**
- Nodes without children get **external nodes** attached, also called NIL.
- All other nodes are called **internal nodes**

nil

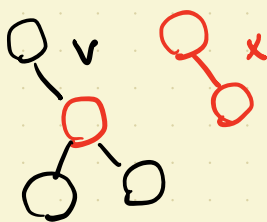
- Rules:
- 1. Every node is either black or red
- 2. The root is black
- 3. Every leaf (external node, or NIL) is black
- 4. If a node is red, then both of its children are black
- 5. For each node, all simple paths from the node to descendant leaves contain **the same number of black nodes** (not counting the node itself)

black height

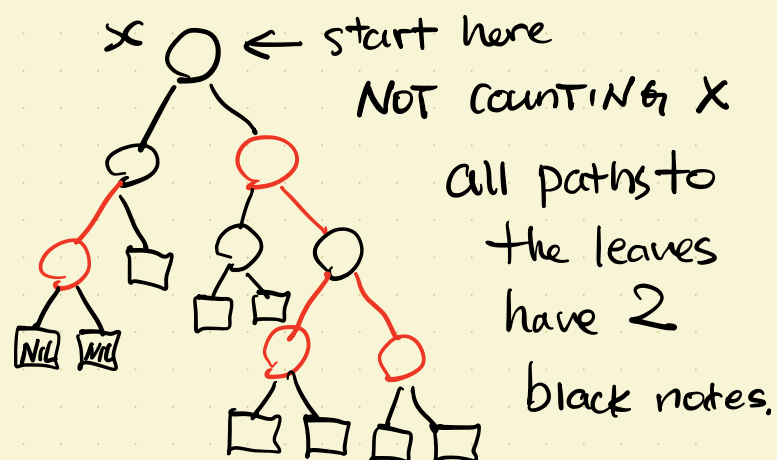
internal node : has key value

external node : has no key value

rule 4.



rule 5.



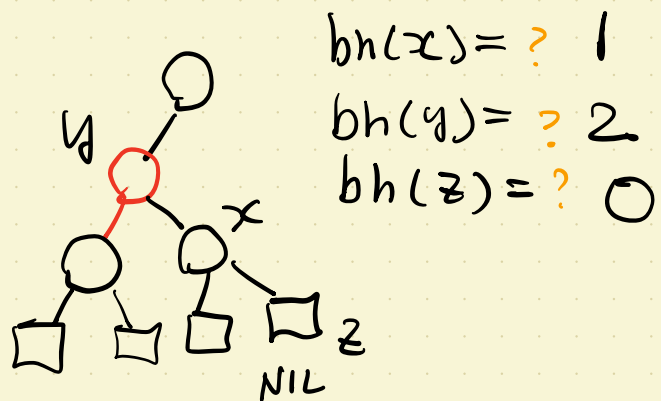
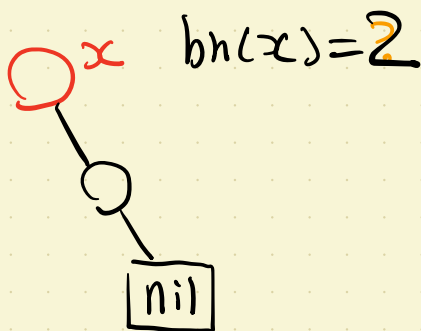
□ = nil
external node

○ or ○ = internal
node

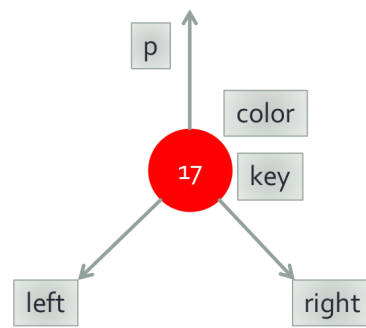
↓
black
height.

Black Height

- Black height:
 $bh(x)$ = number of black nodes on any path from x to a descendant leaf
 (the count is **the same on every path**, so any path works)
- **Does not include node x itself** (regardless of its color)
- The black height of an external node or NIL (leaf) is 0



Representation

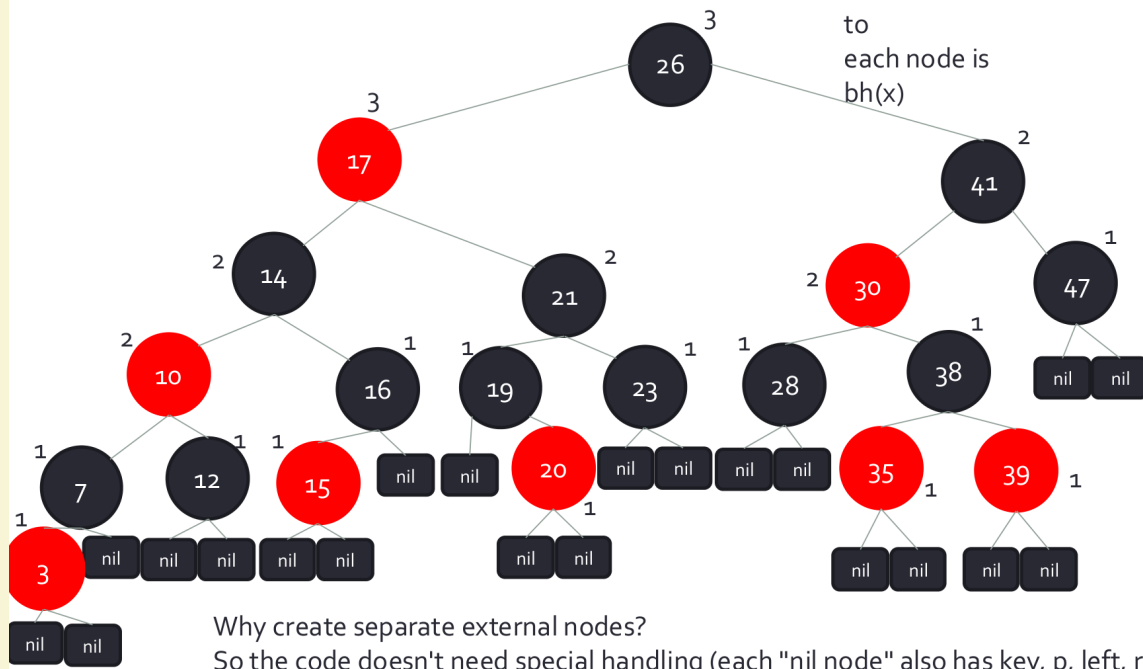


Example C structure

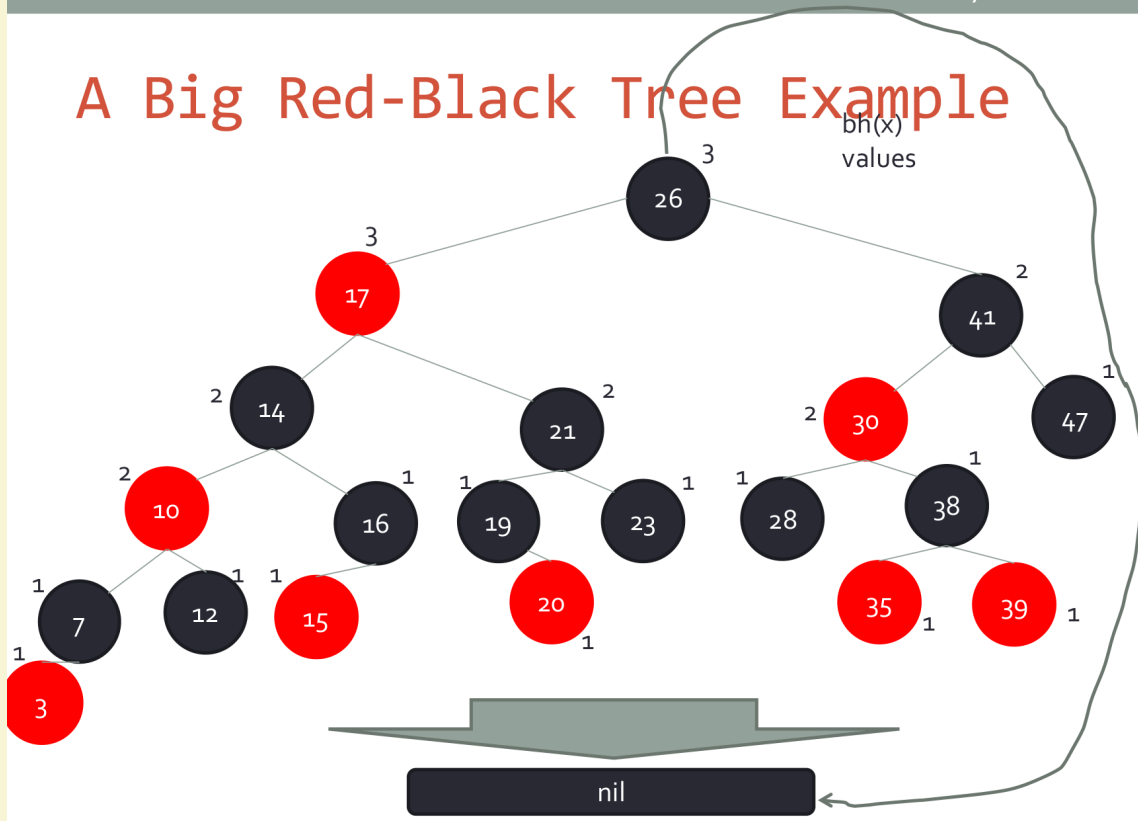
```
typedef enum { RED, BLACK } Color;
```

```
typedef struct RBNode {  
    int key;  
    Color color;  
    struct RBNode *left;  
    struct RBNode *right;  
    struct RBNode *p;  
} RBNode;
```

A Big Red-Black Tree Example



A Big Red-Black Tree Example



```
RBNode *NIL;    /* the one sentinel */
```

```
void rb_init(void) {
    NIL = malloc(sizeof(RBNode));
    NIL->color = BLACK;
    NIL->left = NIL;    /* points to
itself */
    NIL->right = NIL;
    NIL->parent = NIL;
}
```

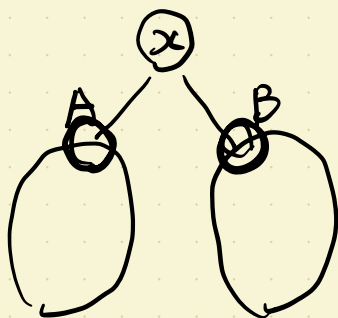
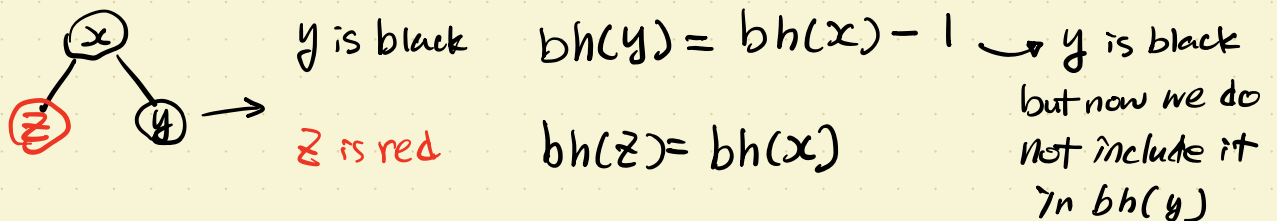
then we can use NIL for any pointer leading
to NIL

- ① root's parent
- ② external nodes

A Bit of Proof

- **Theorem:** A red-black tree with n nodes has height at most $2 \log(n+1)$
- Step 1:
Prove that the subtree rooted at node x has at least $2^{bh(x)} - 1$ internal nodes
- Proof by induction:
 1. When the height of x is 0, then x is a leaf. $bh(x)=0$. $2^0 - 1 = 0$. Indeed the subtree rooted at x has 0 internal nodes.
 2. Assume the height of x is a positive integer, and x is an internal node with two children. Each child's black height is $bh(x)$ (if the child is red) or $bh(x)-1$ (if the child is black)
 3. Assume each child's subtree has at least $2^{bh(x)-1} - 1$ internal nodes
 3. Then the subtree rooted at x has at least $(2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$ (success!) internal nodes

1. x NIL $bh(x)=0$ because "not counting x itself"
2. induction step
Subtree rooted at x , height $\leq k$ has at least $2^{bh(x)} - 1$ internal nodes
evaluating x with height k
 x has a height of $k > 0$



both subtrees have at least $2^{bh(A)} - 1$ internal nodes
 $2^{bh(B)} - 1$ nodes

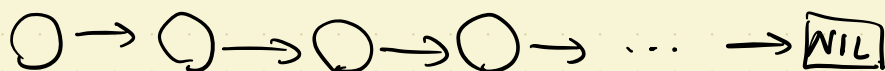
Proving "at least" so we pick the case w/ less nodes.
 $2^{bh(x)-1}$

$$\begin{aligned} \text{the number of internal nodes of subtree rooted at } x &= 1 + 2 \cdot (2^{bh(x)-1} - 1) \\ &= 2^{bh(x)} - 1 \quad \text{step 1} \end{aligned}$$

A Bit of Proof

- 4. If a node is red, then its children are black
- Another way to put it:
- From any node (excluding itself) to a leaf, at least half of the nodes are black
- Let h be the height of the tree, then $bh(\text{root}) \geq \frac{h}{2}$
- "the subtree rooted at node x has at least $2^{bh(x)} - 1$ internal nodes"
- Step 2: How many nodes are there below the root, at minimum?
- Below the root there are at least
- $2^{bh(\text{root})} - 1 \geq 2^{\frac{h}{2}} - 1$ nodes.
- Let the number of nodes be n , then $n \geq 2^{\frac{h}{2}} - 1$
- $h \leq 2 \log(n + 1)$. Yay.

any node



red $\rightarrow$ black $\rightarrow$ red $\rightarrow$ black $\dots$

at least half of the nodes are black
can be more!

height = number of nodes traversed starting from a leaf
(x) excluding the node x

black height: counting only black nodes

$$bh(\text{root}) \geq \frac{h}{2}$$

under root there are at least $2^{bh(\text{root})} - 1$ internal nodes

$$n \geq 2^{bh(\text{root})} - 1$$

$$n+1 \geq 2^{\frac{h}{2}} \geq 2^{\frac{h}{2}} - 1 \quad O(\log n)$$

$$\log(n+1) \geq \frac{h}{2} \quad \log(n+1) \geq \frac{h}{2} \Rightarrow h \leq 2 \log(n+1) \quad \text{\#Step 2}$$

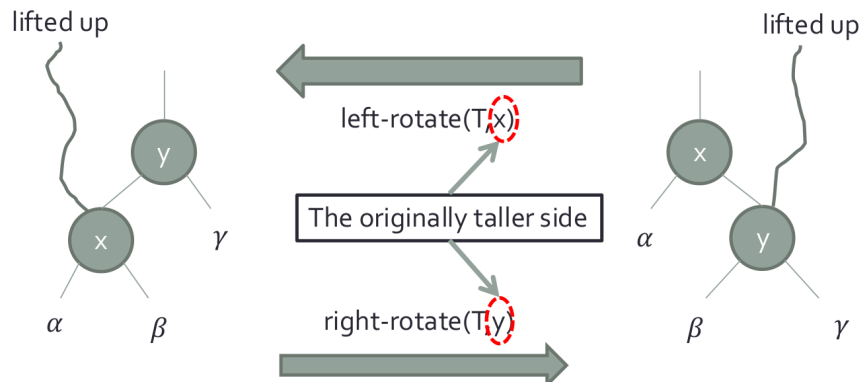
From the proof above...

- Explains why
- 1. Search
- 4. Find max or find min element
- 5. Find the successor or predecessor of an element
- ...can be done in $O(\log n)$

- What about insert and delete?
- Insertion and deletion themselves take $O(\log n)$
- But: after inserting or deleting, the tree may violate red-black properties
- How long does it take to fix? Still $O(\log n)$?

Rotate

- We'll use this in a moment



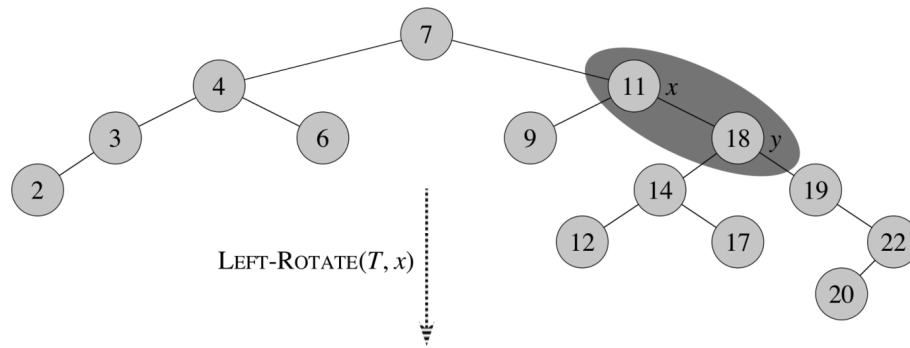
- Note that the ordering is preserved by rotation!
- $\alpha < x < \beta < y < \gamma$

Can be used to adjust the structure of the RB tree

left rotate $\rightarrow$ reduce the height of the right side.

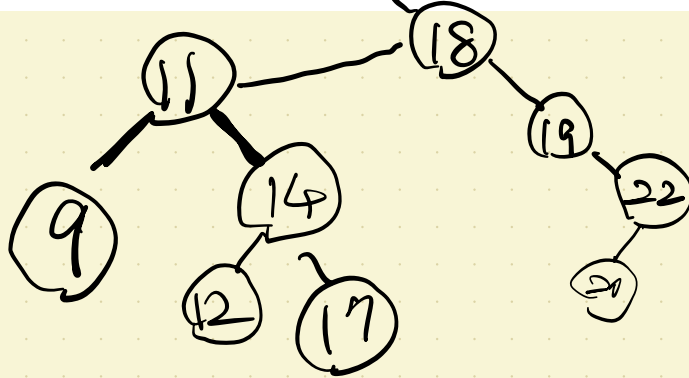
increase the height of the left side.

An Example: Left-Rotate(T, x)



??? ????/

Try!



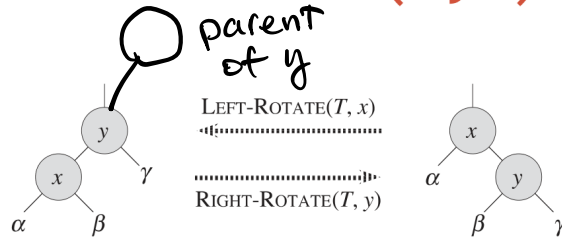
Pseudo-code: Left-Rotate(T, x)

LEFT-ROTATE(T, x)

```

1   $y = x.right$ 
2   $x.right = y.left$ 
3  if  $y.left \neq T.nil$ 
4       $y.left.p = x$ 
5   $y.p = x.p$ 
6  if  $x.p == T.nil$ 
7       $T.root = y$ 
8  elseif  $x == x.p.left$ 
9       $x.p.left = y$ 
10 else  $x.p.right = y$ 
11  $y.left = x$ 
12  $x.p = y$ 

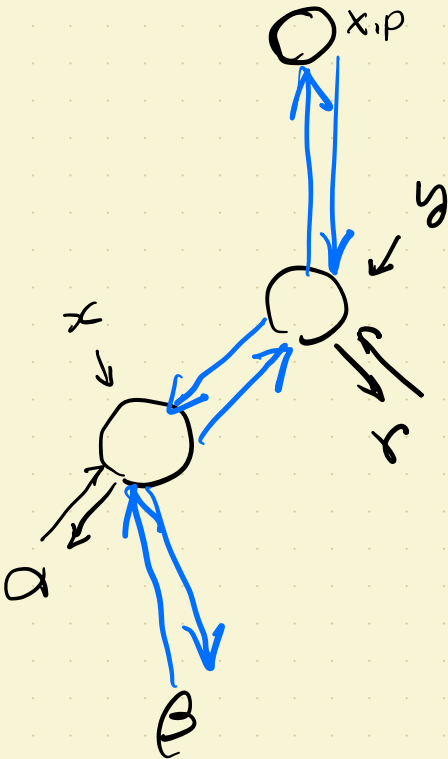
```



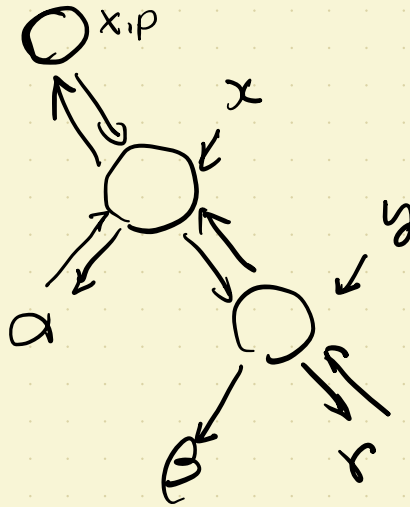
In class wr2:

Can you write the pseudo-code for Right-Rotate(T, y)?

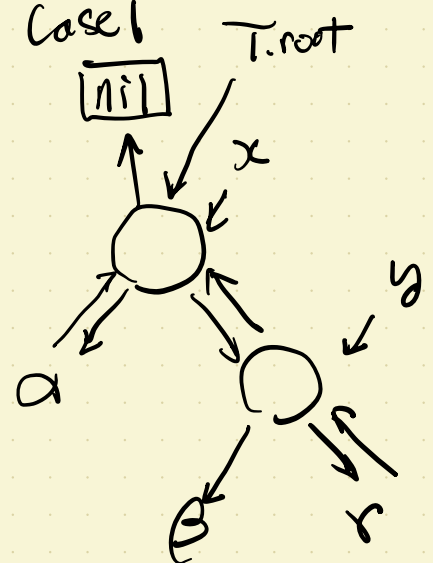
Case 2



Case 3



Case 1

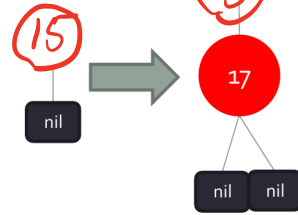


Insertion

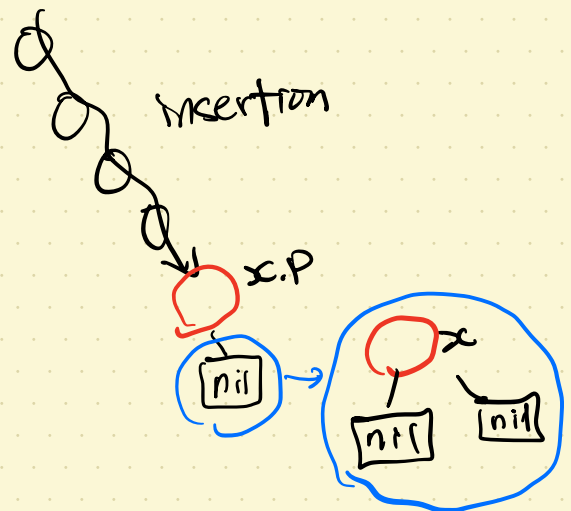
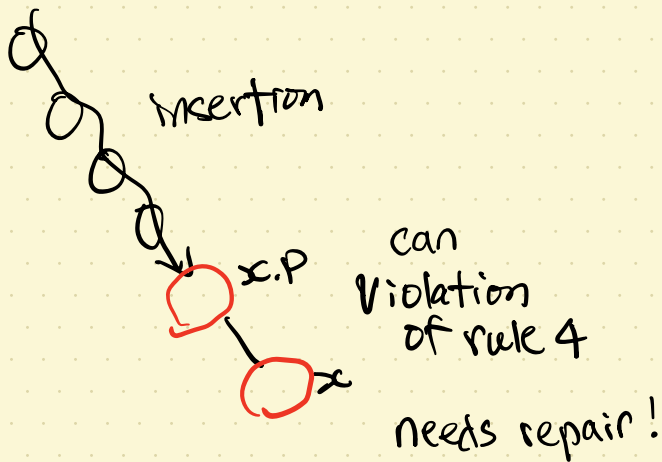
- First, use the standard binary search tree insertion method
- insert(z)
- Differences:
 1. Both of z's children (pointers) point to the `nil` node
 2. z is red
 3. Afterwards we have to fix the parts that violate the red-black tree rules
- How do we fix them?

Which rules might be violated?

- The rules:
 1. Every node is either black or red
 2. The root is black
 3. Every leaf (external node, or nil) is black
 4. If a node is red, then both its children are black
 5. For every node, all paths from it to any descendant leaf contain the same number of black nodes
- Cannot be violated
- Is the whole tree just this newly added node?
- If violated, then z is the root and the whole tree is just z — easy to handle
- The newly added node has two black nil children attached below it
- Is the parent of the newly inserted node also red? Need to handle this!
- For every node, all paths from it to any descendant leaf contain the same number of black nodes
- Not violated, because z is red, z replaces a nil, and z's two children are both nil



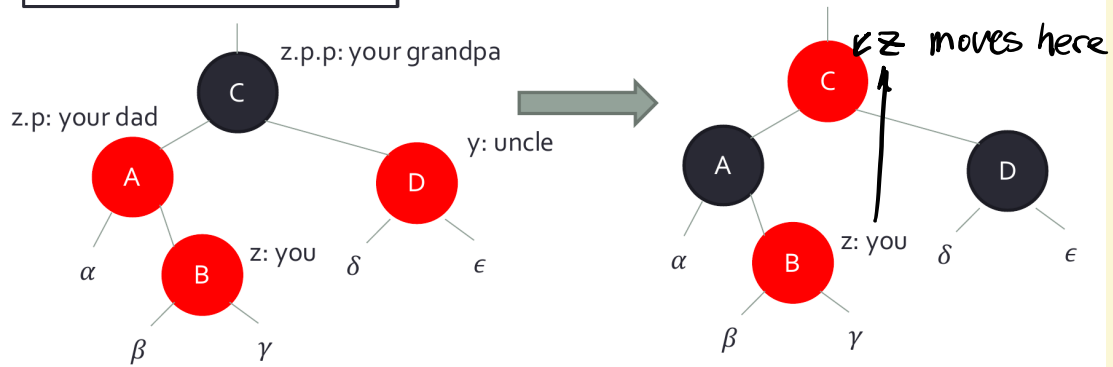
BST insertion



Case 1: Your uncle is red

If you are your dad's **right** child

Continue and check whether new z's dad is red



Pay attention to whether each step preserves the 5th red-black tree property: starting from C and walking to any branch among $\alpha \sim \epsilon$, the number of black nodes along the path must be the same before and after the modification!

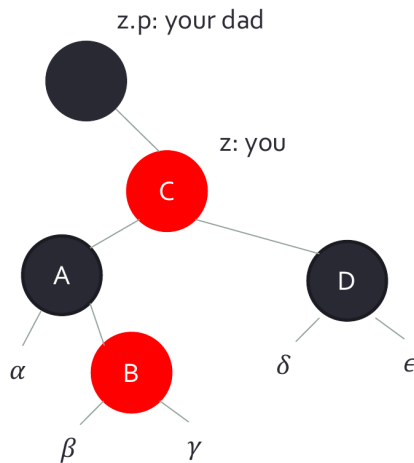
	α	β	γ	δ	ϵ
before	1	1	1	1	1
after	1	1	1	1	1

process from bottom to top

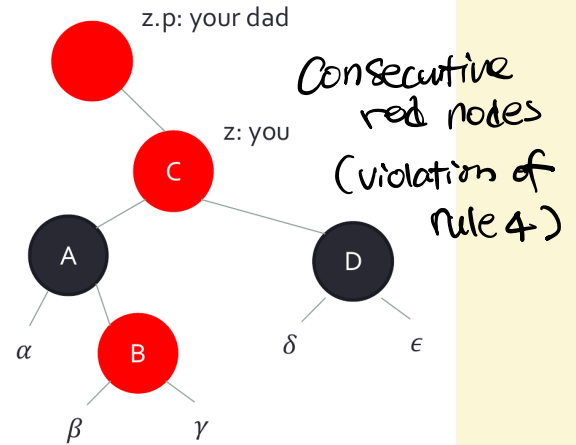
What's shown here is that you are your dad's right child;
you could also be your dad's left child

Continue and check whether new z's dad is red

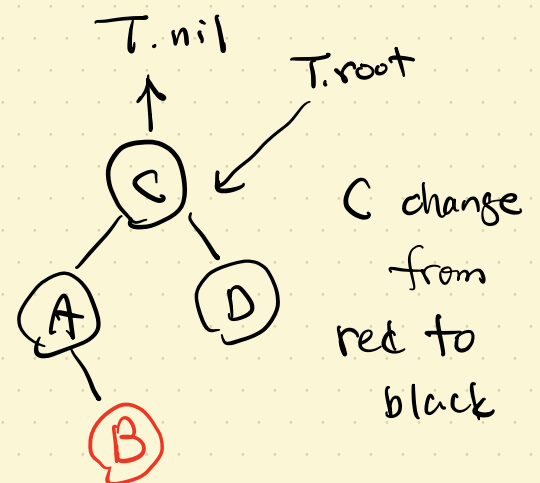
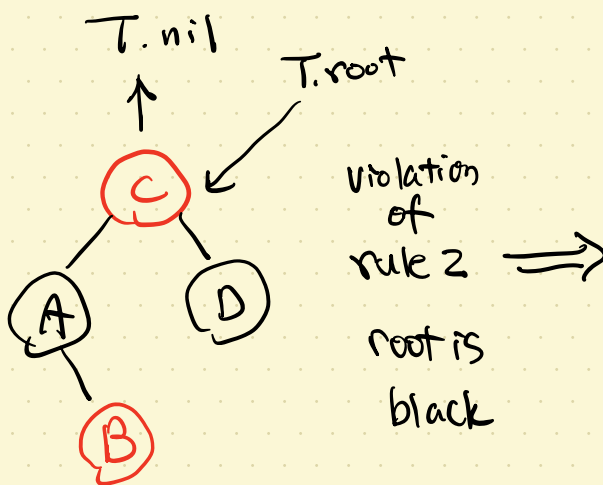
Can stop



Needs to continue



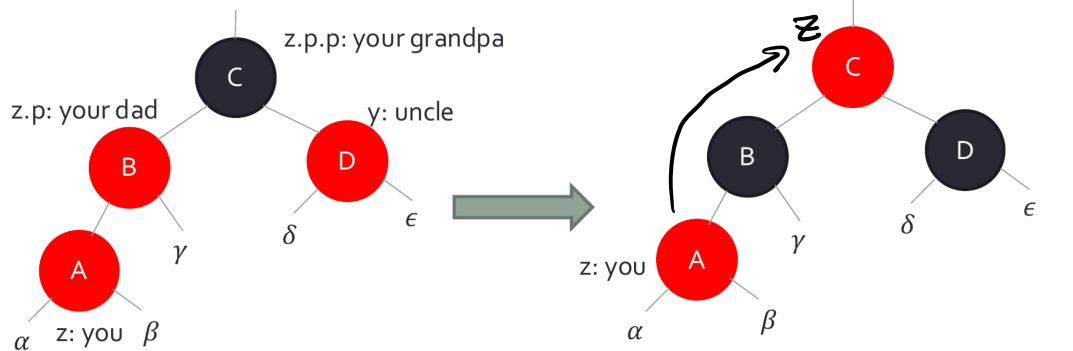
What happens when we reach root?



black height of root +1
but applies to all paths
to the leaves.

Case 1: Your uncle is red

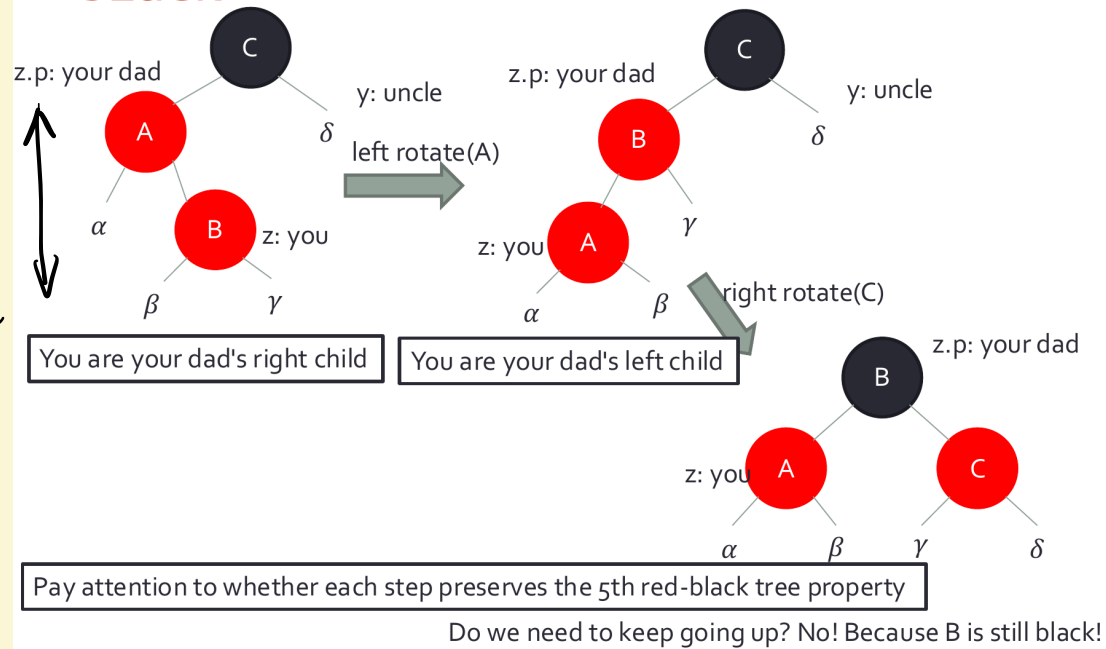
If you are your dad's left child



Pay attention to whether each step preserves the 5th red-black tree property: starting from C and walking to any branch among $\alpha \sim \epsilon$, the number of black nodes along the path must be the same before and after the modification!

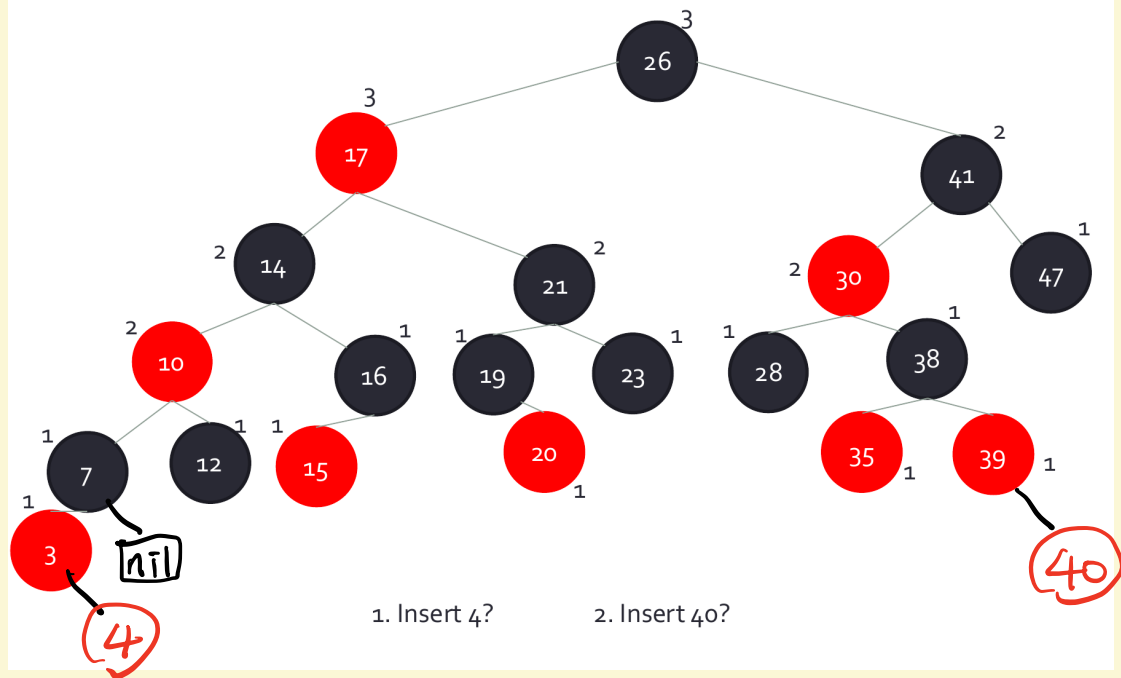
	α	β	γ	δ	ϵ
before					
after					

Cases 2 and 3: Your uncle is black

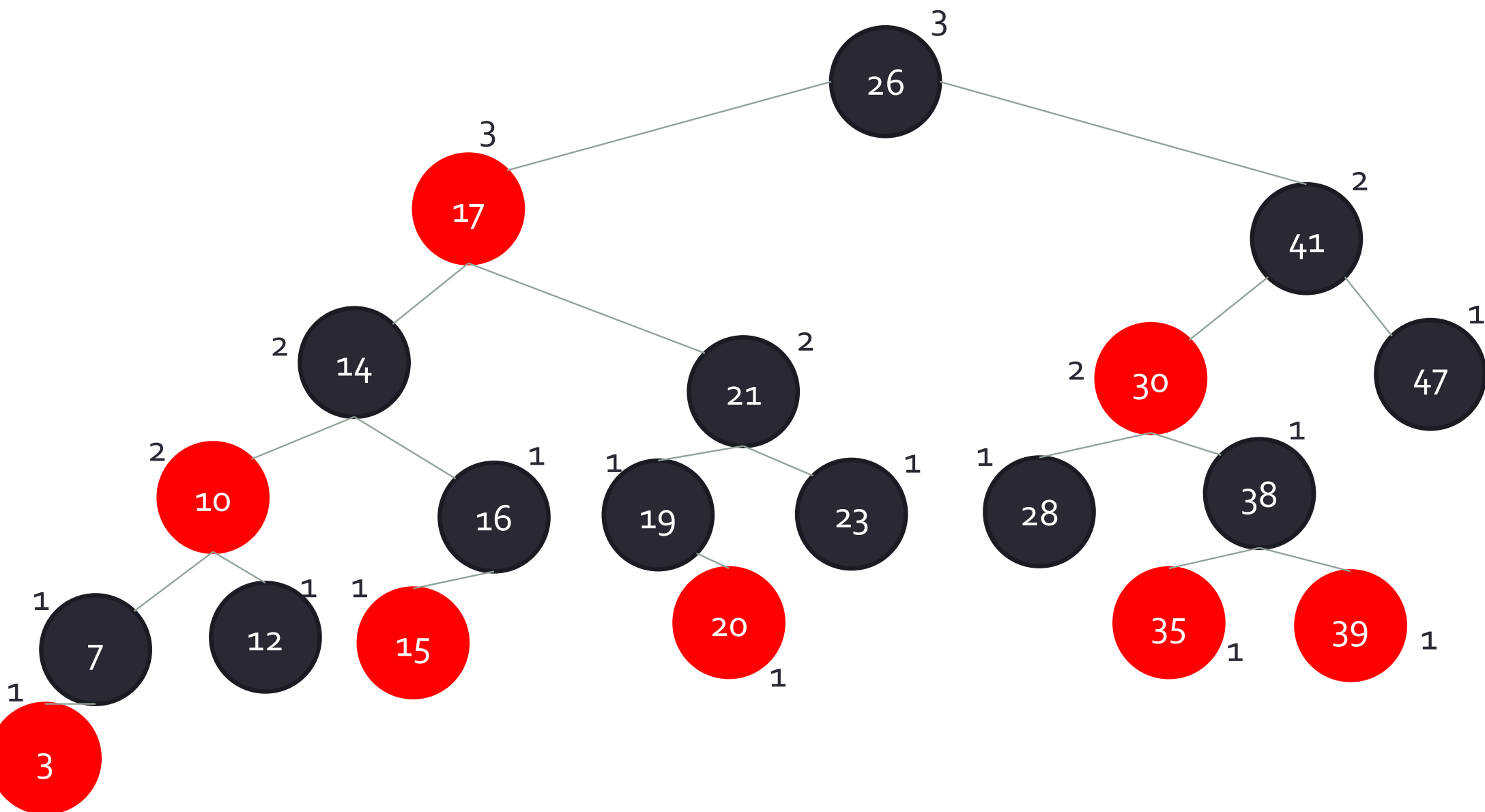


	α	β	γ	δ
before				
step 1				
step 2				

Practice Time



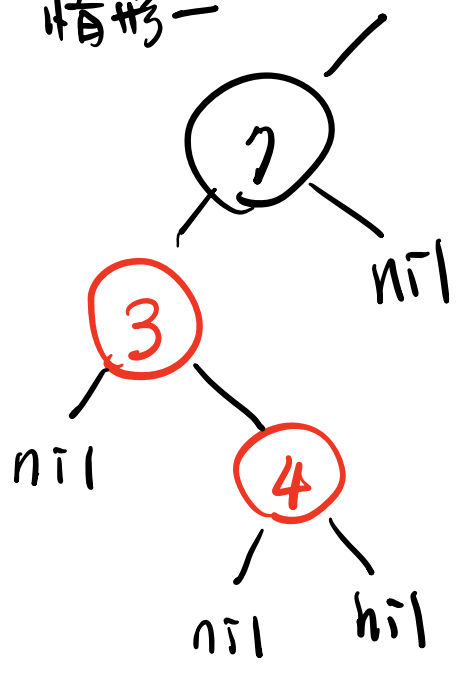
練習時間



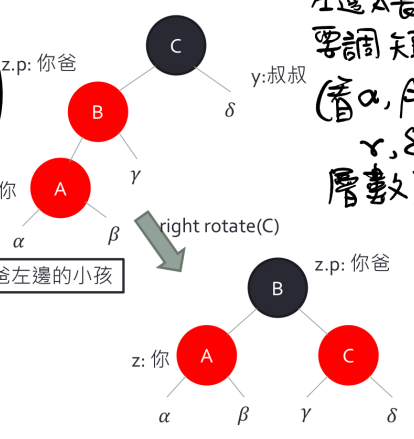
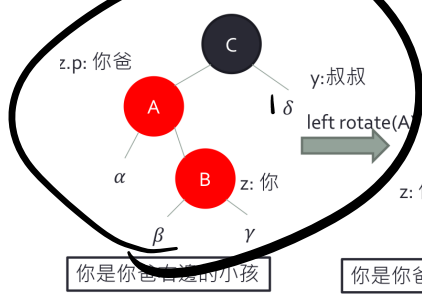
1. 加入₄?

2. 加入₄₀?

情形二



情形二與三：你的叔叔是黑的



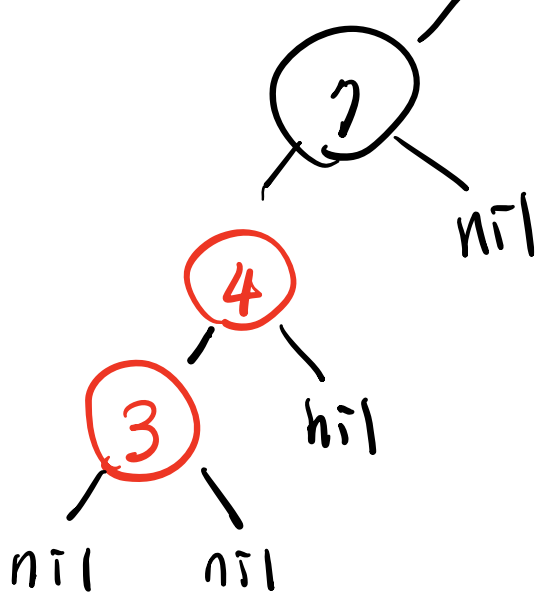
#BN	α	β	γ	δ	ϵ
before					
after					

注意看每一個步驟是否有保持紅黑樹第五個條件

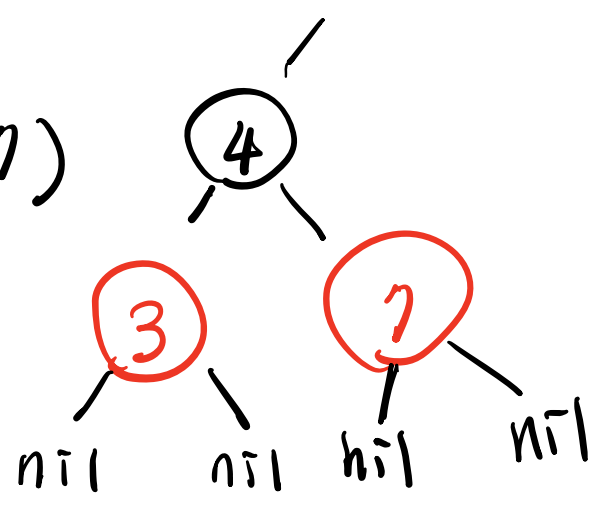
需要繼續往上層看嗎? 不用! 因為B還是黑的!

連續紅色
左邊太長
要調短
(看 α, β
 γ, δ
層數)

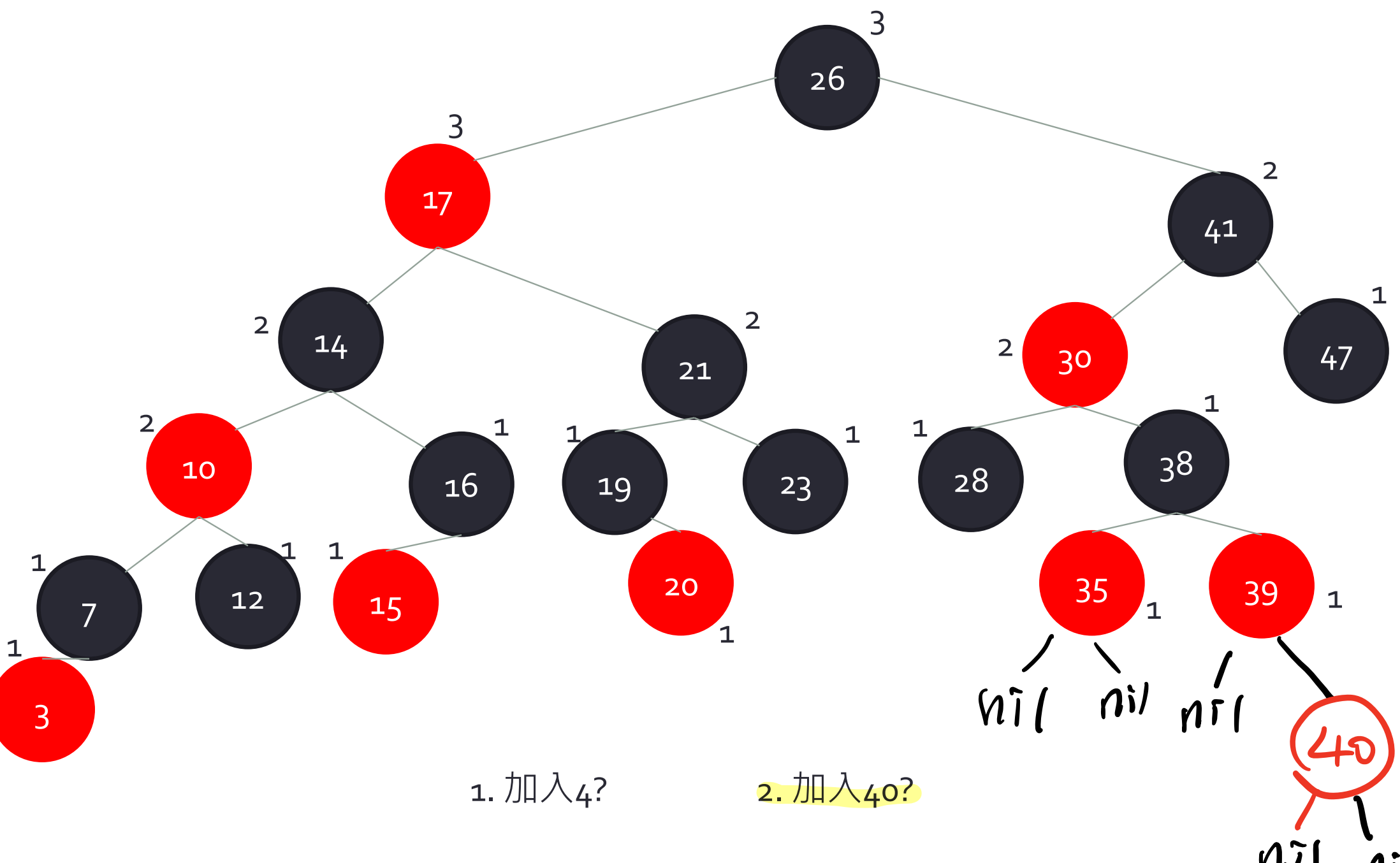
left-rotate(3)



right-rotate(2)



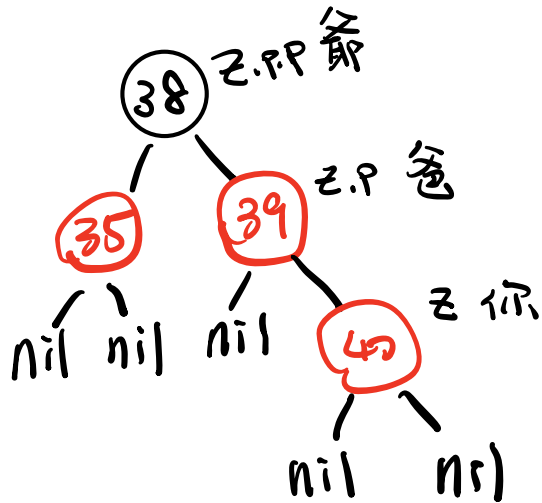
練習時間



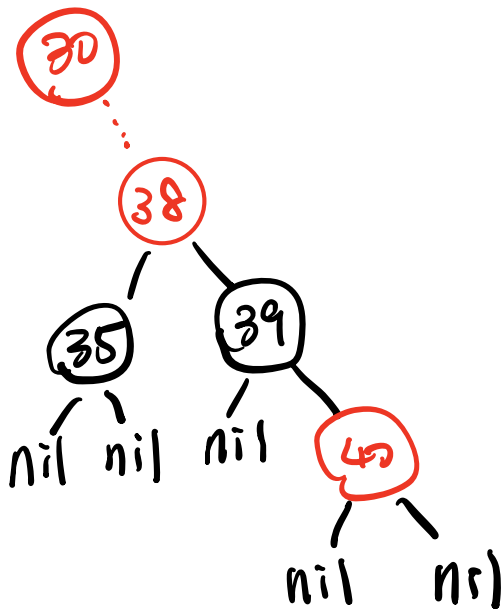
1. 加入4?

2. 加入40?

情形一



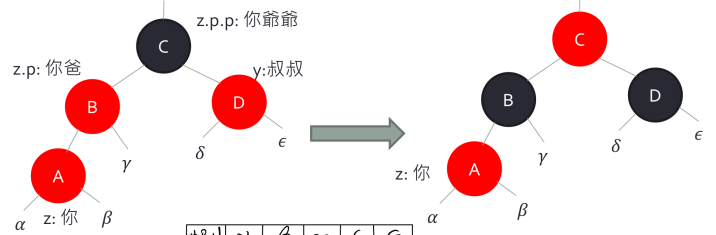
change color



情形一：你的叔叔是紅的

如果你是你爸左邊的小孩

繼續看新z爸爸是不是紅的

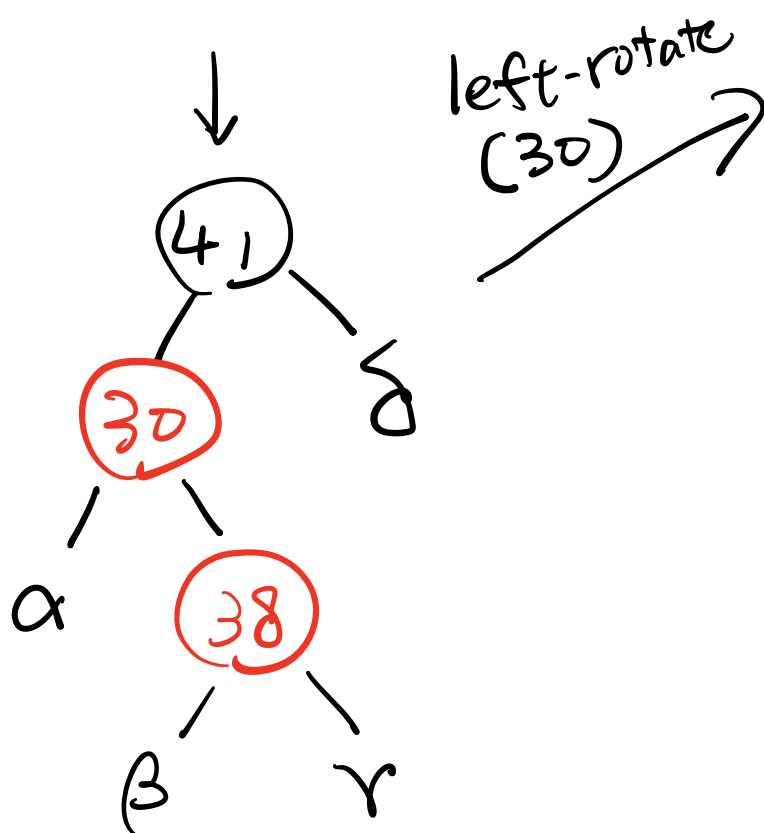
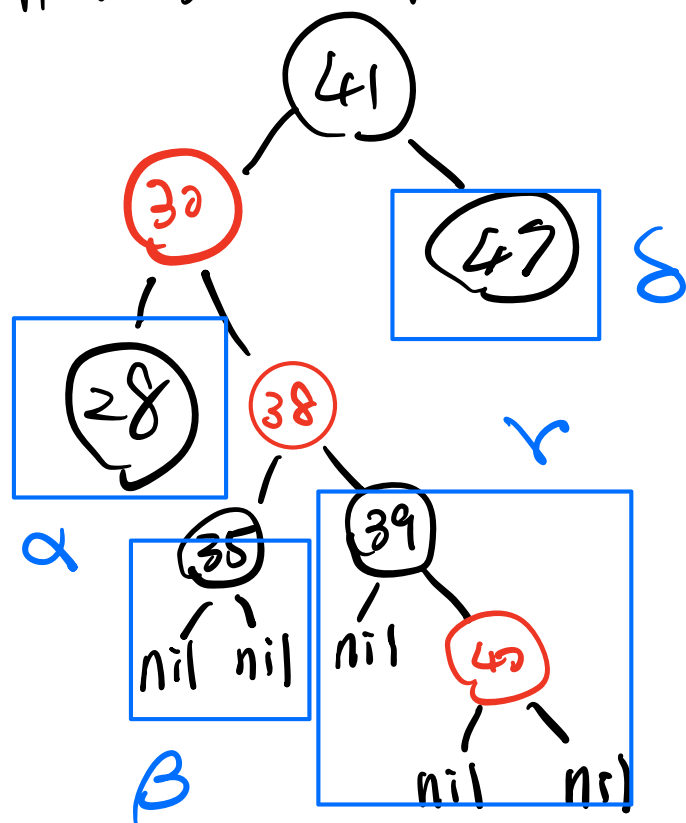


#BN	α	β	γ	δ	ϵ
before					
after					

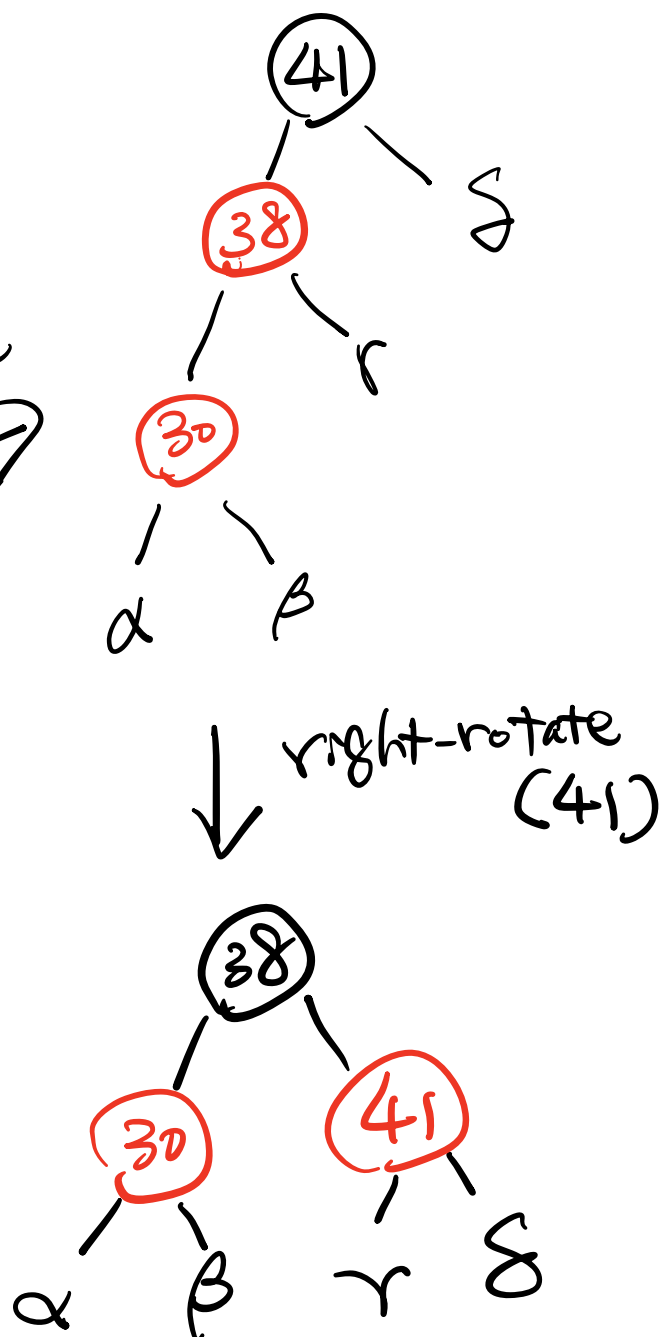
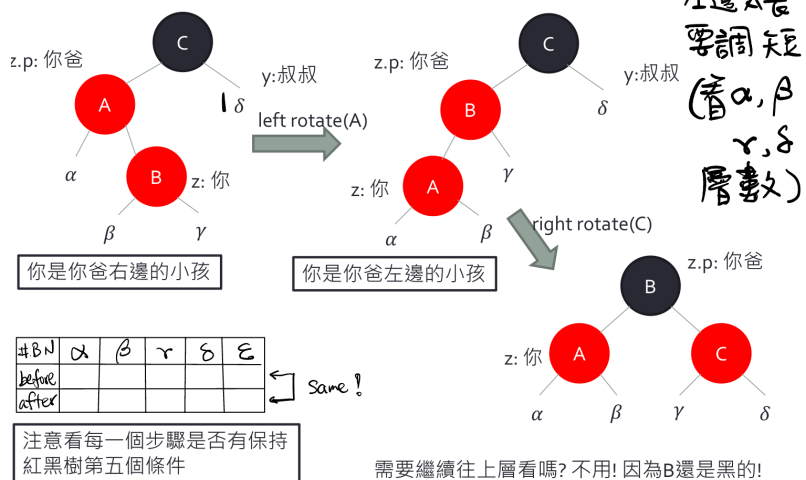
Save!

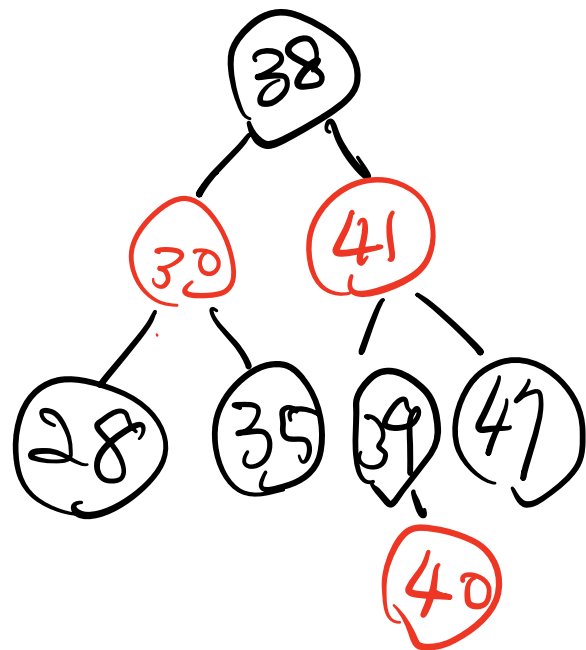
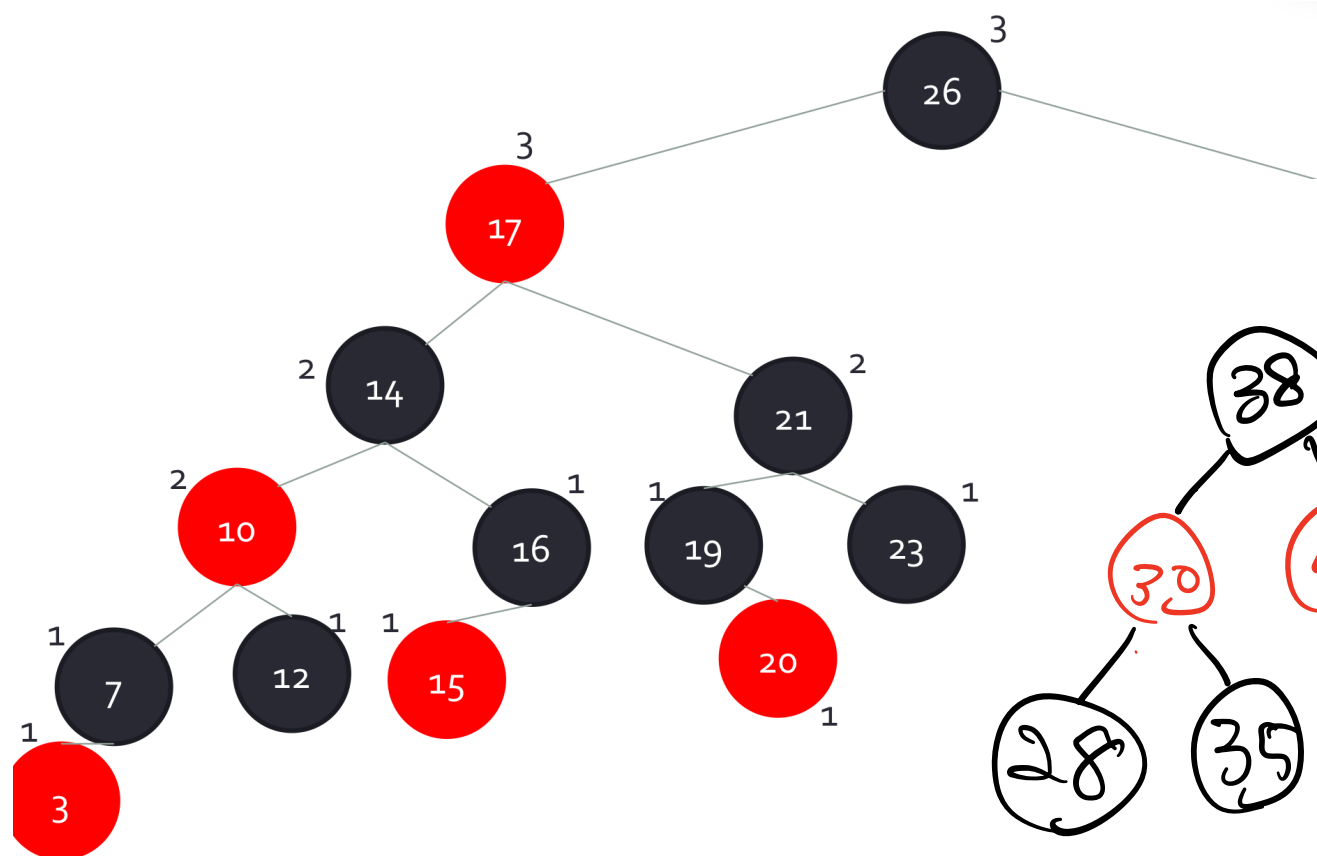
注意看每一個步驟是否有保持紅黑樹第五個條件：
從C開始走到 $\alpha \sim \epsilon$ 任一個分支所經過的黑色node數目在修改前後必須相同！

情形二 $\rightarrow$ 情形三



情形二與三：你的叔叔是黑的





Pseudo-code: Fix R-B tree after insertion

RB-Insert-Fixup(T,z)

While z.p.color==RED (if your dad is red)

if z.p==z.p.p.left (if your dad is your grandpa's left child)

y=z.p.p.right (then your uncle is your grandpa's right child)

if y.color==RED (if uncle is red)

z.p.color=BLACK (set dad to black)

y.color=BLACK (set uncle to black too)

z.p.p.color=RED (set grandpa to red)

z=z.p.p (move yourself up to grandpa)

else if z==z.p.right (if you are your dad's right child)

z=z.p (move yourself up to your dad)

LEFT-ROTATE(T,z)

z.p.color=BLACK (set your dad to black)

z.p.p.color=RED (set your grandpa to red)

RIGHT-ROTATE(T,z.p.p)

else if z.p==z.p.p.right (if your dad is your grandpa's right child, same as above, just swap left and right)

T.root.color=BLACK

Case 1

Case 2

Case 3

Cases 4-6: mirror situations

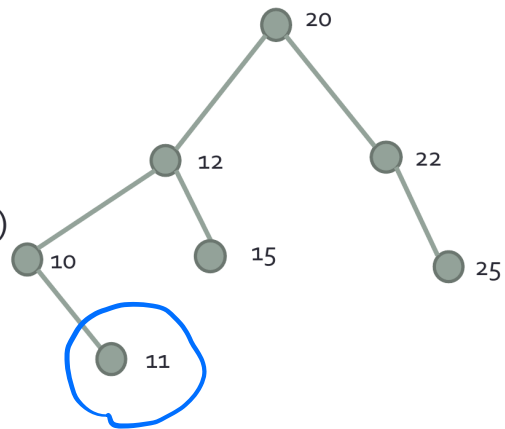
Write the pseudo code of the second part.
Case 4,5,6

How much time does it take?

- The normal binary tree insert itself takes $O(\log n)$
- Because the maximum height is $2 \log(n+1)$
- What about the time spent on fixup?
- Worst case? Case 1 keeps happening, each time z moves up two levels
- Worst-case running time is proportional to the height
- Also $O(\log n)$
- What if cases 2 or 3 happen?
- Done after one execution. $O(1)$. (So smaller than $O(\log n)$)
- Also, at most 2 rotations are needed. (Case 2 or 3 will not happen again)
- For the correctness proof see Cormen ~~pages 3-5~~

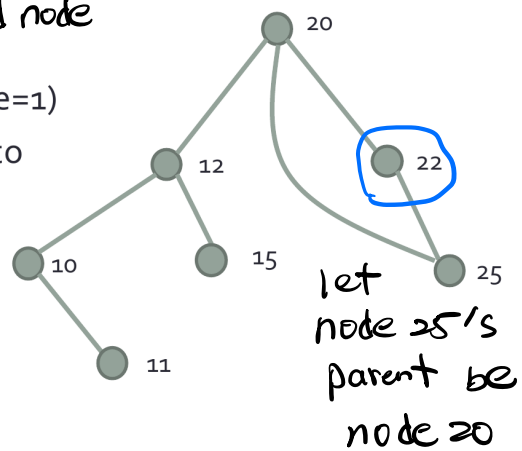
How do we delete a node? (binary search tree review)

- First we need to find that node
- Then, several different cases:
 - no child node
↑
- If there are no hands (degree=0)
- Just take it out directly



How do we delete a node? (binary search tree review)

- If there is only one hand (degree=1)
 - Then bring that single child up to attach to the parent
- Example: delete 22



How do we delete a node? (binary search tree review)

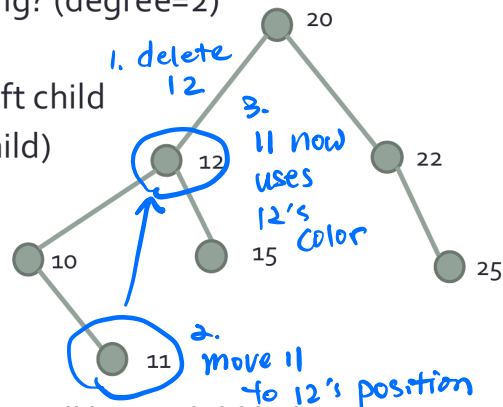
- What if both hands have something? (degree=2)

- For example, delete 12

- Find the largest node under the left child

- (or the smallest under the right child)

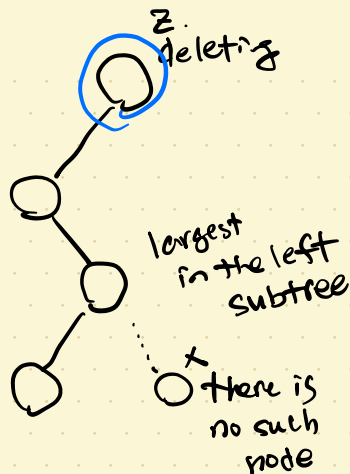
- Remove it and move it to the position of the originally deleted node



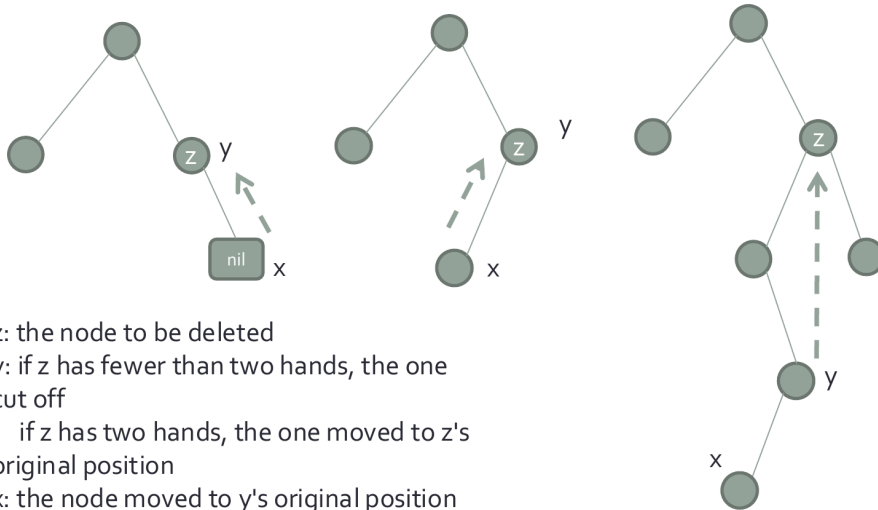
- Question: what if that largest node still has a child below it?

- Just bring it up directly (at most one left hand)

- At this point, the moved-up node takes on the color of the node originally at the new position



Definitions of x, y, z



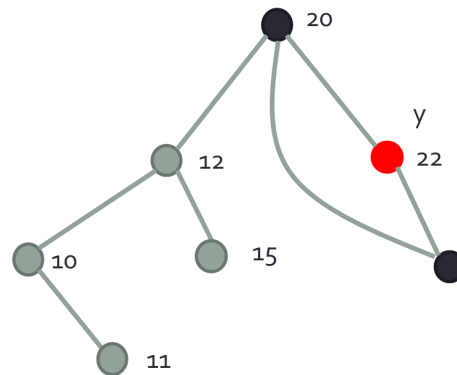
z: the node to be deleted
 y: if z has fewer than two hands, the one cut off
 if z has two hands, the one moved to z's original position
 x: the node moved to y's original position

We will remember y's color.

y's color is removed from the tree.

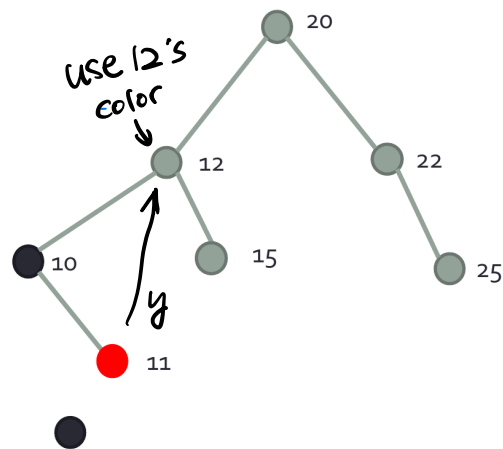
After removing a node, when are the rules violated?

- Suppose, among the three cases above,
- the node that is moved (both hands) or deleted (one or zero hands) is y
- When y is red (its original color), will moving or deleting it violate any rule?
- 1. Black height will not change, because y is a red node
- 2. Could it cause two red nodes to be adjacent (parent-child)?
- A. If y is being deleted: because y is red, both the level above and below are black
- No problem



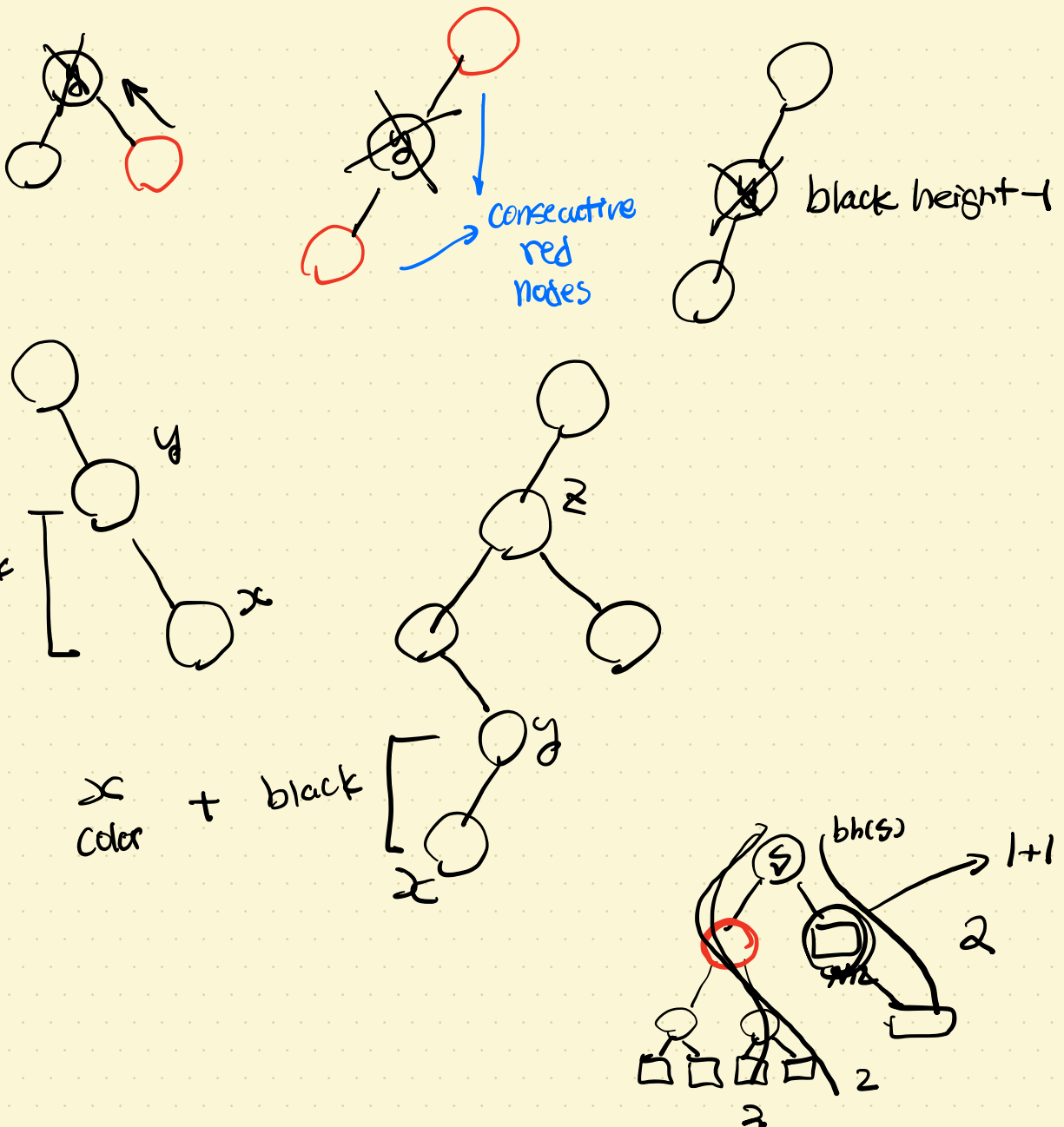
After removing a node, when are the rules violated?

- B. If y is being moved, and y's children are also black, no problem is caused
- 3. If y is red, it cannot be the root, so it won't cause a violation of "root must be black" either
- Combining the three points above, only when y is black can problems arise; then we need to fix them



Possible rule-violation cases

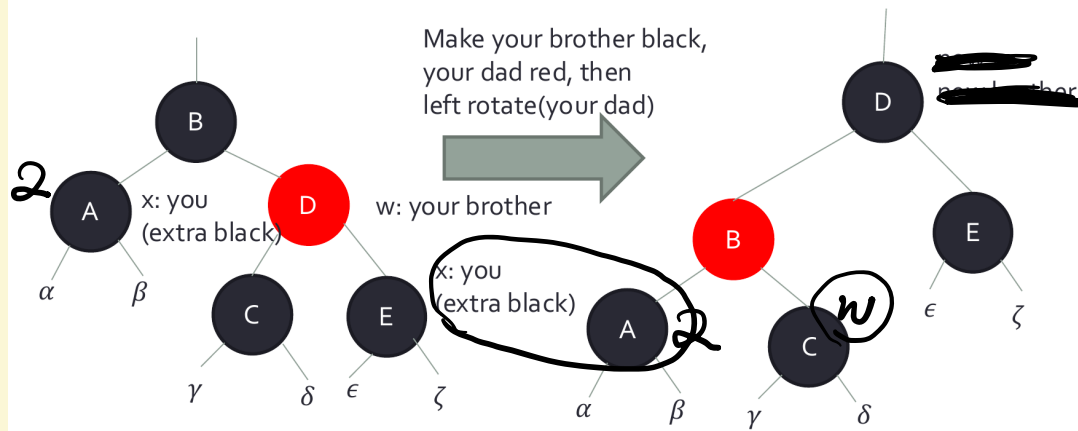
- When y is black, the following cases may violate the rules:
 1. y is the root. After y is deleted, one of its children is red and becomes the root
 2. The level above and below y were both red; after moving or deleting, two reds become adjacent
 3. After y is deleted or moved, y 's ancestors end up with inconsistent black heights (because y was black)
- After y is removed, y 's "blackness" gets "pushed" into x
 - x means: this position needs an extra unit of black
 - But x itself may originally be red or black



Is x "red-and-black" or "black-and-black"?

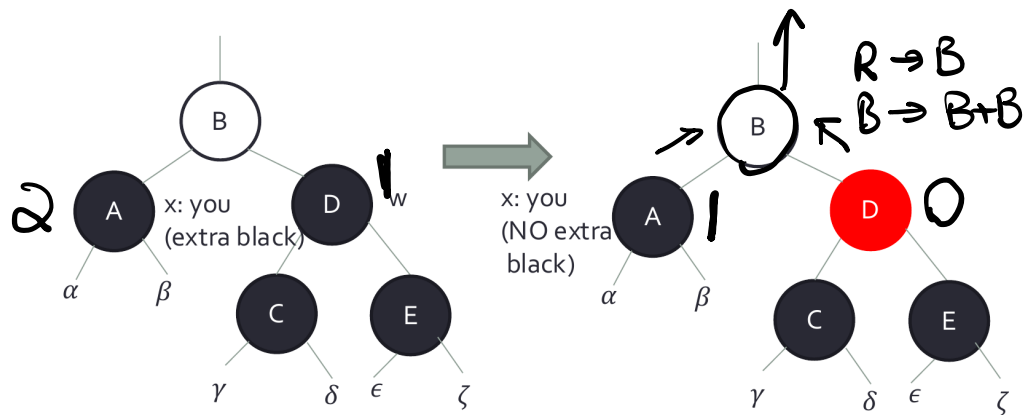
- 1. If x is "red-and-black", we just change it to black.
 - (We need an extra unit of black here, so changing red to black supplies that extra unit)
- 2. If x is the root and "black-and-black", we can simply change it to plain black
 - (Since it's already at the root, even if we remove a unit of black no ancestor will be short a black node as a result)
- 3. Otherwise, handle with the following four cases... (push the black-and-black toward the root)

Case 1: Your brother is red

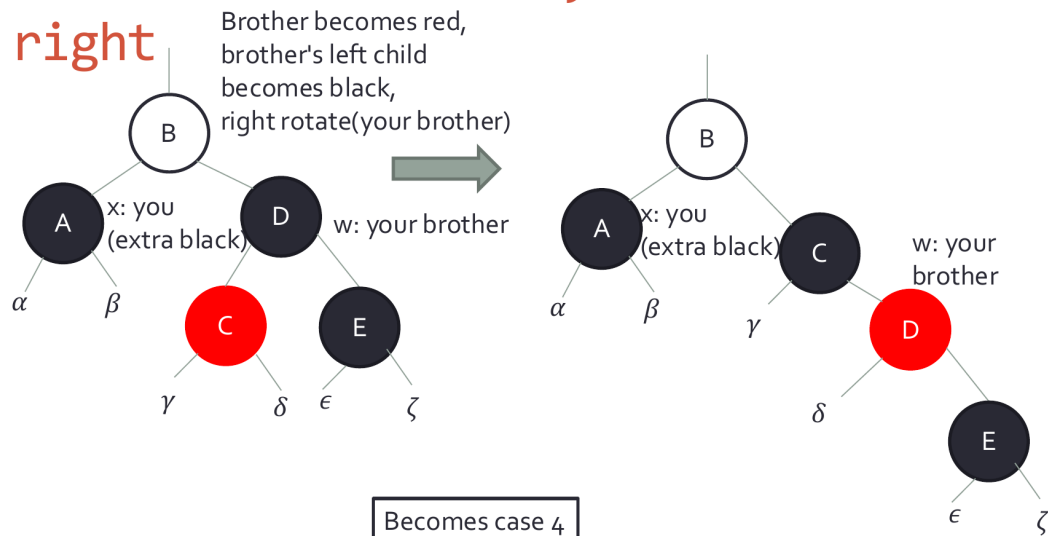


	α	β
before	3	3
after	3	3

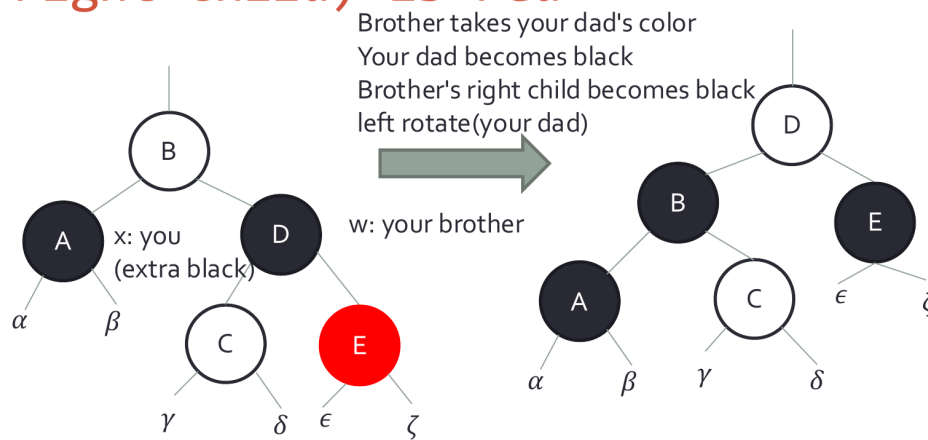
Case 2: Your brother is black & your nephews are all black



Case 3: Your brother is black & your nephews (brother's children) are red on the left, black on the right



Case 4: Your brother is black & your right nephew (brother's right child) is red



Pseudo-code: Fix R-B tree after deletion

RB-Delete-Fixup(T, x)

while $x \neq T.root$ && $x.color == BLACK$ (you are not root and are black)

if ($x == x.p.left$) (you are your dad's left child)

$w = x.p.right$ (your brother is your dad's right child)

 if $w.color == RED$ (if your brother is red)

$w.color = BLACK$ (set your brother to black)

$x.p.color = RED$ (set your dad to red)

 LEFT-ROTATE($T, x.p$)

$w = x.p.right$ (your new brother is now your dad's right child)

 if $w.left.color == BLACK$ && $w.right.color == BLACK$ (both nephews are black)

$w.color = RED$ (set brother to red)

$x = x.p$ (you become your dad)

 else if $w.right.color == BLACK$ (your nephews: right black, left red)

$w.left.color = BLACK$ (set left nephew to black)

$w.color = RED$ (set brother to red)

 RIGHT-ROTATE(T, w)

$w = x.p.right$ (new brother is now your dad's right child)

$w.color = x.p.color$ (set brother's color to dad's color)

$x.p.color = BLACK$ (set dad to black)

$w.right.color = BLACK$ (set right nephew to black)

 LEFT-ROTATE($T, x.p$)

$x = T.root$ (jump straight to root, ready to exit)

 else ($x == x.p.right$) (if you are your dad's right child, same as above with left/right swapped)

~~$x.color = BLACK$ (if we exit the loop — x is red+black, or root is black+black — just set it to a single black)~~

Case 1

Case 2

Case 3

Case 4

Cases 5-8

How much time does the post-Delete adjustment take?

- Resolution path map:
- $1 \rightarrow 2 \rightarrow$ solved (going from 1 to 2: in case 2 the B in the figure must originally be red, so we can resolve directly)
- $1 \rightarrow 3 \rightarrow 4 \rightarrow$ solved
- $1 \rightarrow 4 \rightarrow$ solved
- $3 \rightarrow 4 \rightarrow$ solved
- $4 \rightarrow$ solved
- $2 \rightarrow$ solved (if B in the figure was originally red), or go to 1 or 2 or 3 or 4 (x moves up one level) (if B in the figure was originally black)
- So worst case is 2, 2, 2, ... all the way up to the root
- $O(\log n)$

More reading...

- Cormen ch.13
- You might be interested in some other balanced binary trees:
 - AVL Tree (Cormen problem 13-3)
 - 2-3 Trees
 - B-trees, generalization of 2-3 trees (Cormen ch. 18)